Model Compression and Input Optimization Co-Design for Edge Driver Monitoring Systems

1st Oumar Mamoun Ibrahim
Department of Computer Engineering
University of Sharjah
Sharjah, UAE
U22200741@sharjah.ac.ae

2nd Mohamad Khairi bin Ishak
Department of Computer Engineering
University of Sharjah
Sharjah, UAE
mishak@sharjah.ac.ae

3rd Ahmed Atef Abouelhamd
Department of Computer Engineering
University of Sharjah
Sharjah, UAE
U23104122@sharjah.ac.ae

***Abstract*—Edge driver monitoring systems (DMS) must detect distraction and drowsiness in real time on small compute and memory budgets. We study whether model compression and input preprocessing can be co-designed for this task without violating a safety recall floor. On the Driver Monitoring Dataset with a subject-disjoint split, we combine structured ℓ_1-norm channel pruning and 8-bit integer (INT8) quantization on YOLO11n and YOLO26n with three input methods (resize, static crop, and overlapping collage) from 640 to 80 pixels, and we score every configuration by macro-recall on the same frames and boxes over three seeds. YOLO11n holds the 5-point recall floor up to 70% sparsity, and quantization-aware training (QAT) brings this model to 1.29 MB at 84.8% macro-recall against 86.4% for the baseline. The overlapping collage gives the highest recall at every resolution down to 160 pixels. The cheapest configuration that holds the floor is the 160-pixel collage with 30% sparsity and INT8 post-training quantization (PTQ), which needs $24.5\times$ less compute and $4.8\times$ less storage but passes only narrowly. The 320-pixel collage with INT8 gives the highest recall, 92.9%, at $4.0\times$ less compute. Heavier pruning at reduced input did not transfer from validation to test drivers.**

***Index Terms*—Object Detection, Driver Monitoring Systems, Edge Deployment, Model Pruning, Quantization**

I. Introduction

Most computer vision models are designed for high-performance systems, which increases neural network size and the computation and storage required for inference [1]. This makes direct deployment on edge devices impractical [2], [3]. To bridge this gap, practitioners adapt models to constrained hardware through pruning [4], quantization [5], and knowledge distillation [6]. These methods reduce parameter count and inference cost from different angles. Edge-deployed driver monitoring systems (DMS) are particularly demanding, because the detector must evaluate every frame in real time to catch distraction and drowsiness cues.

This work addresses one research question:

- How do model-level compression and input-level preprocessing compare in isolation, and can their co-design reduce edge cost without violating a safety recall limit?

We approach the question from the model side and the input side, then combine both in one co-designed pipeline. On the model side, we evaluate structured ℓ_1-norm channel pruning, which removes the weakest convolutional channels [8]. Structured pruning reduces floating-point operations (FLOPs) on commodity hardware and needs no sparse accelerator. We also evaluate post-training quantization (PTQ) and quantization-aware training (QAT), which convert 32-bit floating-point (FP32) networks to 8-bit integers (INT8). On the input side, we evaluate a static 400×400 region of interest (RoI) crop and a static overlapping collage across input resolutions of 640, 320, 160, and 80 pixels.

TABLE I
Coverage of prior work versus this paper

Work	Pruning	Quant.	RoI	DMS	Joint
Filter/channel pruning [8]–[11]	✓	–	–	–	–
Deep compression [7]	✓	✓	–	–	–
Quantization [12], [13]	–	✓	–	–	–
Tiling / multi-scale RoI [18], [19]	–	–	✓	–	–
DMS / distraction [20]–[22]	–	–	–	✓	–
This work	✓	✓	✓	✓	✓

The main contributions of this paper are:

- A model-input co-design framework that searches input method, resolution, sparsity, and quantization together under one macro-recall floor, using validation data only.
- An evaluation protocol that scores every configuration on the same frames and ground-truth boxes at one threshold, so an object removed by the input step counts as a missed detection.
- A multi-seed study of ℓ_1-norm pruning, INT8 PTQ, and true fake-quantized QAT on YOLO11n and YOLO26n, with clip-level confidence intervals for the small test set.
- A measured account of why INT8 detection recall becomes unstable when the output layers of the detection head are quantized, and how keeping them in floating point removes the instability.

The code, configurations, and per-run results that reproduce every table and figure are available at https://github.com/IamOumarIbrahim/pruning-missed-detection.

II. Related Work

A. Model Compression for Edge Object Detection

Deploying detectors on embedded hardware relies on pruning, quantization, and efficient architecture design. Unstructured pruning removes individual low-magnitude weights [7],

but the resulting sparsity seldom yields real speedups on commodity accelerators. Structured pruning removes entire filters or channels and produces a smaller dense network. Li *et al.* [8] rank filters by their ℓ_1 norm. He *et al.* [9] select channels through LASSO regression with feature-map reconstruction. Liu *et al.* [10] exploit sparsity in batch-normalization scale factors. Fang *et al.* [11] extend structural pruning to arbitrary architectures through dependency graphs, which matters for YOLO models whose concatenations and residual links couple channels across layers.

Quantization lowers numerical precision to cut memory traffic and latency. Jacob *et al.* [12] introduced integer-arithmetic-only inference with QAT, and Krishnamoorthi [13] compared PTQ and QAT for INT8 deployment. Han *et al.* [7] showed that pruning and quantization are complementary when applied jointly.

The YOLO family is a standard choice for real-time detection on constrained devices. YOLOv7 introduced trainable bag-of-freebies and planned re-parameterization [14]. YOLOv10 removed non-maximum suppression (NMS) through consistent dual assignments [15]. YOLO11 introduced C3k2 and C2PSA blocks [16]. YOLO26 adds NMS-free inference by default and removes distribution focal loss [17]. Compression studies of these detectors typically fix the input, compress the network, and report mean average precision (mAP) on general-purpose benchmarks. Recall on safety-critical minority classes receives far less attention.

B. Input-Level and RoI Optimization

Input resolution is a primary driver of inference cost, so cropping, tiling, and resolution selection are natural levers for real-time systems. Unel *et al.* [18] showed that tiling improves small-object detection. Najibi *et al.* [19] proposed AutoFocus, which processes only selected high-resolution regions. These methods target generic scenes in which object locations are unpredictable. In an in-cabin camera, the driver occupies a predictable region of the frame, which makes a static RoI a natural design choice.

C. Driver Monitoring Systems

Deep learning has become the standard approach to driver distraction and drowsiness detection. The Driver Monitoring Dataset (DMD) [20] provides multi-modal in-cabin recordings with annotated attention and alertness states. Abouelnaga *et al.* [21] studied real-time distracted driver posture classification and treated inference speed as a design constraint. Kashevnik *et al.* [22] surveyed driver distraction detection methods and organized them into a common framework.

D. Research Gap

Existing work treats compression, input-level optimization, and driver monitoring largely in isolation (Table I). Three gaps stand out:

- YOLO compression studies optimize mAP on general benchmarks. Recall on safety-critical minority classes receives little attention.
- Static RoI strategies have not been compared systematically across input resolutions for in-cabin behavior detection.
- No prior work has selected pruning, quantization, and input preprocessing jointly under a recall constraint for DMS.

III. Methodology

A. Proposed Co-Design Framework

Existing YOLO compression work fixes the input and compresses the network. Our framework differs in three ways. First, the input and the model are one search space: a configuration c is a tuple of input method, resolution, sparsity, and quantization. Second, the input comes first. Each model is trained on the input it will see, and pruning starts from that checkpoint, not from a full-frame model. Third, input loss and model loss share one denominator: every configuration is scored on the same frames and boxes, so an object the crop removes is a miss.

The study has three phases. Phase I measures pruning and quantization alone. Phase II measures input methods alone. Phase III searches their combinations and selects the cheapest configuration that holds the recall floor on validation data:

$$c^* = \arg\min_{c} \ \mathrm{GFLOPs}(c) \quad \text{s.t.} \quad R^{\mathrm{val}}(c) \geq R^{\mathrm{val}}_{\mathrm{base}} - \delta, \quad (1)$$

where R^{val} is validation macro-recall, $R^{\mathrm{val}}_{\mathrm{base}}$ is that of the FP32 baseline, and $\delta = 0.05$ is the safety margin defined in (4).

B. Models and Dataset

YOLO11n has 2.59M parameters and 6.50 GFLOPs at 640×640 and integrates C3k2 blocks with C2PSA spatial attention. Phase I also evaluates YOLO26n, with 2.51M parameters and 5.90 GFLOPs, which is NMS-free and has no distribution focal loss. DMD contains 1280×720 RGB in-cabin video of drivers performing drowsy and distracted behaviors. We extract 15,723 frames at 1 FPS from 14 subjects and center-crop them to 640×640. Two annotators labeled each frame. Negative frames have no box. Positive frames have one box from four classes: yawning, hand-over-mouth, drinking, or phone use. Negative frames form $\approx 81\%$ of all frames (12,722), and phone use forms $\approx 81\%$ of positive frames (2,437).

Subjects are split 8:3:3 so that no driver appears in two splits. This tests generalization to new drivers, which random frame splits cannot do because consecutive frames of one driver are nearly identical. Table II gives the composition of each split and how it is used. Training subjects fit all weights, including fine-tuning after pruning, QAT, and PTQ range calibration. Validation subjects select the best checkpoint of each run and the final configuration in (1). Test subjects are used once, for the reported numbers.

Baselines start from COCO-pretrained weights and train for 100 epochs in Ultralytics 8.4 on an NVIDIA RTX 4060 8 GB GPU with batch size 16 and three seeds. The automatic optimizer setting resolves to MuSGD with learning rate 0.01, momentum 0.9, and weight decay 0.0005.

TABLE II
SUBJECT-DISJOINT SPLITS AND THEIR USE

	Train	Validation	Test
Subjects	8	3	3
Subject IDs	1, 4, 6–9, 13, 14	2, 3, 11	5, 10, 12
Frames	9,087	3,423	3,213
Clips	40	15	13
Yawning frames (clips)	94 (10)	32 (5)	33 (5)
Hand-over-mouth frames (clips)	83 (10)	30 (4)	28 (5)
Drinking frames (clips)	154 (8)	54 (3)	56 (3)
Phone-use frames (clips)	1,417 (9)	523 (3)	497 (3)
Weights, fine-tuning, QAT	✓	–	–
PTQ calibration (300 frames)	✓	–	–
Best checkpoint per run	–	✓	–
Configuration selection	–	✓	–
Reported results	–	–	✓

C. Evaluation Protocol

All phases use one protocol. Detections are matched to ground truth greedily by confidence, within a class, at an intersection over union (IoU) of 0.5. All metrics use a fixed confidence threshold of $\tau = 0.5$. A validation-tuned threshold is reported only as a sensitivity check. Precision P_c and recall R_c of class c give the class score in (2):

$$F1_c = \frac{2\,P_c\,R_c}{P_c + R_c}. \tag{2}$$

Macro metrics in (3) are unweighted means over the four classes:

$$R = \frac{1}{4}\sum_{c=1}^{4} R_c, \quad P = \frac{1}{4}\sum_{c=1}^{4} P_c, \quad F1 = \frac{1}{4}\sum_{c=1}^{4} F1_c. \tag{3}$$

Macro-recall R from (3) is the primary safety metric, because automotive DMS guidance treats a missed distraction or drowsiness event as a hazard and a false alarm as an annoyance [23], [24]. A configuration is safe if it holds the floor

$$R \geq R_{\text{floor}} = R_{\text{base}} - \delta, \quad \delta = 0.05, \tag{4}$$

where R_{base} is the macro-recall of the unpruned FP32 model of the same architecture at 640×640. Cost-normalized efficiency is defined in (5):

$$\text{Eff}_{\text{FLOPs}} = \frac{R\,(\%)}{\text{GFLOPs}}, \quad \text{Eff}_{\text{MB}} = \frac{R\,(\%)}{\text{Size (MB)}}. \tag{5}$$

Size is always the ONNX file. Mean average precision (mAP) uses all detections down to a confidence of 0.001.

Detections from a crop or collage are mapped back to the 640×640 frame. Each ground-truth box is matched against the part that stays visible, and a box with no visible part remains in the denominator as a miss. An object shown twice by overlapping tiles counts once for recall and adds no false positive.

Results are the mean $\pm$ sample standard deviation over three seeds. The test set has only 33 yawning and 28 hand-over-mouth frames, so one missed frame moves class recall by 3.0 to 3.6 percentage points (pp). Frames of one clip are also correlated, and those classes each come from only five clips. We therefore report 95% confidence intervals from a bootstrap that resamples whole clips, keeps seeds together, and compares candidate and baseline on the same resampled clips.

D. Model-Side Optimization

Pruning. We prune in one shot, from the trained baseline of the same seed, at target sparsities of 10% to 90% in 10-point steps. C3k2 blocks are first split into an equivalent two-branch form so that their branches can be pruned independently. A dependency graph [11] groups coupled channels. Each group is ranked by the ℓ_1 norm of its filters, and the lowest-ranked target fraction of that group is removed. Ranking is local to each group, not global across the network. The detection head, C2PSA attention, and convolutions with fewer than 8 output channels are excluded. Every pruned model is fine-tuned before evaluation for 100 epochs with the baseline training settings. Target sparsity applies to channels: 70% removes about 91% of the parameters of the pruned stages, but only 65.6% of the total, because the head and C2PSA are excluded.

Post-training quantization. We export each FP32 model to ONNX (opset 17) and apply static QDQ quantization with ONNX Runtime 1.30. Weights use signed 8-bit symmetric per-channel scales. Activations use unsigned 8-bit asymmetric scales. Ranges are a moving average (constant 0.01) of per-frame minima and maxima over 300 fixed frames from the training subjects. The frames are sampled at random, so 246 are negatives and the rarest classes appear only twice. The output convolution of each head branch and the box and class decoding remain in floating point.

Quantization-aware training. We insert fake-quantization modules with NVIDIA ModelOpt 0.48 through Ultralytics. Weights use signed 8-bit symmetric per-channel scales and activations use signed 8-bit symmetric scales. Ranges are calibrated once on eight training batches and then frozen. The same head layers stay in floating point. Each FP32 model is fine-tuned for five epochs with SGD at a learning rate of 0.001, without mixed precision. We then store the weights as INT8 and keep the learned scales. Both methods therefore yield comparable INT8 files.

E. Input-Side Optimization

Each input method is applied to the 640×640 frame and resized bilinearly to 640, 320, 160, or 80 pixels. Models are trained on each input from COCO-pretrained weights. YOLO requires input sides divisible by its maximum stride of 32, so the 80-pixel input runs at 96×96.

Crop. The RoI is a fixed 400×400 window at $x \in [140, 540]$, $y \in [120, 520]$. It keeps 39% of the pixels. We chose it from the driver position in the camera view and checked it against training annotations (Table III). Every yawning box stays at least 50% visible and 97% stay complete, so the mouth region survives. Hand-over-mouth boxes are large, with a median height of 405 pixels, so few fit whole, but all stay at least 50% visible. The crop also magnifies

TABLE III
SHARE OF BOXES (%) KEPT BY EACH INPUT METHOD

	Train				Test			
	Crop		Collage		Crop		Collage	
Class	Full	≥50%	Full	≥50%	Full	≥50%	Full	≥50%
Yawning	97	100	100	100	100	100	100	100
Hand-over-mouth	5	100	82	100	0	100	75	100
Drinking	0	53	9	92	16	73	27	100
Phone use	6	67	53	99	3	64	34	93

what it keeps: at a 320-pixel input, the median short side of a yawning box grows from 35 to 55 pixels. The cost falls on peripheral cues. Drinking and phone-use boxes extend toward the door and the center console, and test drivers hold the phone differently from training drivers, so 9.3% of test phone-use boxes leave the crop entirely.

Collage. The collage rebuilds the frame on a 640×640 canvas from three tiles: a 320×640 head strip at $x \in [173, 493]$, and 320×320 left and right boxes at $y \in [215, 535]$. It keeps the head at full height and both hand zones, and removes only the frame corners. The tiles overlap in 25% of the canvas area, so that a hand near the face appears in two tiles. Table III shows the cost on the training subjects: at least 92% of boxes stay half visible in one tile, but only 9% of drinking boxes and 53% of phone-use boxes fit whole, so tile seams cut objects. Fig. 1 shows both methods.

F. Joint Co-Design

Phase III evaluates the full grid of 24 INT8 candidates: the three input methods at 320 and 160 pixels, each at 0%, 30%, 50%, and 70% sparsity. Each candidate starts from the Phase II checkpoint trained on its input, is pruned as in Section III-D, fine-tuned for up to 50 epochs with a patience of 10, and quantized. Fine-tuning is shorter than in Phase I because the source model has already converged on the same input. The selection rule (1) picks the candidate on validation data, and test results are reported once. Because validation is small, we also rank all candidates by compute, then memory, and recommend the first whose mean test macro-recall holds the floor in (4). Its paired difference from the baseline, ΔR, and the clip-bootstrap interval are reported with it.

IV. RESULTS AND DISCUSSION

A. Phase I: Model-Side Optimization

Tables IV and V and Figs. 2 and 3 report Phase I. Four findings stand out.

- *Pruning headroom.* YOLO11n holds its 81.4% floor up to 70% sparsity ($85.6 \pm 1.4\%$ against $86.4 \pm 1.3\%$), then falls to 79.8% at 80% and 62.3% at 90%. YOLO26n starts higher under the pinned toolchain ($89.0 \pm 1.3\%$, floor 84.0%) and holds the floor up to 60% sparsity ($84.1 \pm 3.9\%$).
- *Class sensitivity.* Yawning, the smallest object, fails first. YOLO11n yawning recall falls from 80.8% at baseline to 61.6% at 80% sparsity and 16.2% at 90%. Macro-precision stays between 82.9% and 87.4%, so pruning costs recall rather than precision.
- *Cost.* At 70% sparsity YOLO11n has 0.89M parameters, 2.50 GFLOPs, and a 3.65 MB FP32 ONNX file, against 2.59M, 6.50 GFLOPs, and 10.12 MB. Parameters fall by only 65.6% because the excluded head grows from 16.7% to 48.4% of the remaining weights.

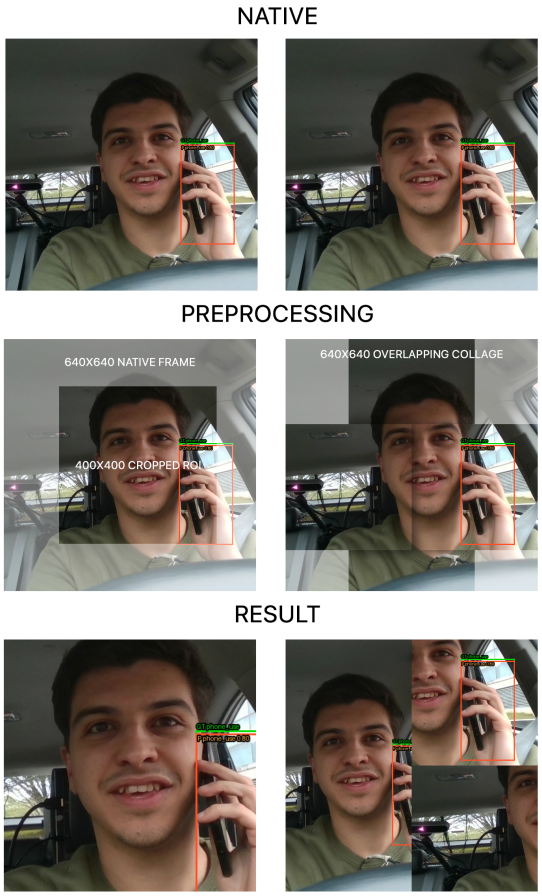


Fig. 1. Native frames and the output of each input method

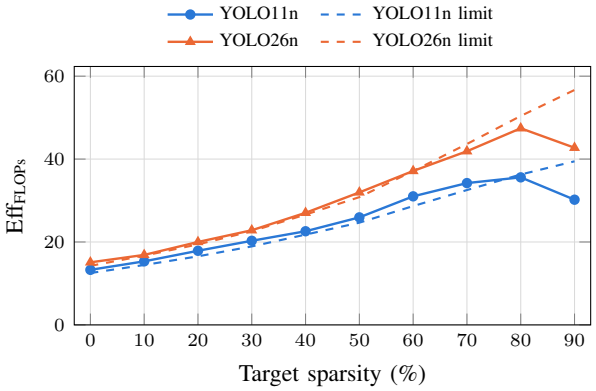


Fig. 2. FP32 efficiency against target sparsity. A solid line below its dashed limit fails the floor

TABLE IV
FP32 PRUNING RESULTS ON THE TEST SET AT $\tau = 0.5$

	Cost				Class recall (%)						
Sparsity	Params (M)	GFLOPs	Size (MB)	P (%)	Yawning	Hand-mouth	Drinking	Phone use	R (%)	$\text{Eff}_{\text{FLOPs}}$	Safe
YOLO11n											
Base	2.59	6.50	10.12	85.2 ± 4.6	80.8 ± 1.7	92.9 ± 3.6	89.3 ± 1.8	82.6 ± 1.5	86.4 ± 1.3	13.3	✓
10%	2.24	5.63	8.79	87.4 ± 2.6	80.8 ± 6.3	90.5 ± 4.1	94.0 ± 4.1	79.9 ± 3.4	86.3 ± 3.7	15.3	✓
20%	1.94	4.92	7.66	86.1 ± 1.7	83.8 ± 7.6	95.2 ± 2.1	90.5 ± 4.1	82.4 ± 1.4	88.0 ± 2.9	17.9	✓
30%	1.67	4.30	6.61	86.5 ± 2.4	84.8 ± 8.0	92.9 ± 0.0	90.5 ± 2.7	80.7 ± 2.7	87.2 ± 2.1	20.3	✓
40%	1.43	3.74	5.70	86.5 ± 1.1	76.8 ± 6.3	90.5 ± 5.5	89.3 ± 3.1	81.2 ± 1.2	84.4 ± 2.3	22.6	✓
50%	1.23	3.30	4.94	85.4 ± 4.0	78.8 ± 5.2	92.9 ± 3.6	89.3 ± 0.0	81.5 ± 0.9	85.6 ± 2.3	25.9	✓
60%	1.04	2.84	4.22	84.5 ± 2.0	85.9 ± 7.0	92.9 ± 6.2	91.1 ± 4.7	82.5 ± 1.2	88.1 ± 1.2	31.0	✓
70%	0.89	2.50	3.65	84.0 ± 6.1	79.8 ± 6.3	89.3 ± 3.6	89.9 ± 2.7	83.4 ± 1.6	85.6 ± 1.4	34.2	✓
80%	0.77	2.24	3.17	82.9 ± 2.7	61.6 ± 9.3	85.7 ± 6.2	88.1 ± 1.0	83.9 ± 3.3	79.8 ± 1.8	35.6	×
90%	0.68	2.06	2.83	83.4 ± 2.8	16.2 ± 1.7	78.6 ± 6.2	78.6 ± 3.1	75.8 ± 1.9	62.3 ± 1.7	30.2	×
YOLO26n											
Base	2.51	5.90	9.34	88.5 ± 1.1	83.8 ± 6.3	95.2 ± 2.1	93.5 ± 2.1	83.5 ± 1.2	89.0 ± 1.3	15.1	✓
10%	2.18	5.03	8.12	89.7 ± 1.8	72.7 ± 10.9	92.9 ± 0.0	92.3 ± 2.7	81.8 ± 1.1	84.9 ± 3.0	16.9	✓
20%	1.91	4.33	7.08	88.0 ± 1.4	75.8 ± 3.0	92.9 ± 3.6	92.3 ± 3.7	85.1 ± 1.2	86.5 ± 2.5	20.0	✓
30%	1.66	3.72	6.11	89.7 ± 2.9	76.8 ± 4.6	90.5 ± 2.1	92.9 ± 0.0	79.3 ± 3.1	84.9 ± 1.9	22.8	✓
40%	1.43	3.16	5.27	87.3 ± 2.3	76.8 ± 4.6	91.7 ± 5.5	92.3 ± 5.5	80.8 ± 2.8	85.4 ± 4.2	27.0	✓
50%	1.25	2.73	4.56	88.9 ± 1.6	79.8 ± 6.3	96.4 ± 0.0	89.9 ± 5.5	82.4 ± 1.8	87.1 ± 1.5	32.0	✓
60%	1.07	2.26	3.86	87.7 ± 3.7	70.7 ± 7.6	92.9 ± 3.6	89.9 ± 5.5	82.8 ± 3.7	84.1 ± 3.9	37.1	✓
70%	0.92	1.93	3.32	88.1 ± 3.0	65.7 ± 3.5	84.5 ± 5.5	86.3 ± 2.1	86.0 ± 1.5	80.6 ± 0.7	41.9	×
80%	0.80	1.67	2.85	85.8 ± 1.6	63.6 ± 10.9	82.1 ± 3.6	86.3 ± 2.7	84.1 ± 1.6	79.0 ± 1.9	47.4	×
90%	0.71	1.48	2.50	86.2 ± 7.6	25.3 ± 25.4	71.4 ± 15.6	79.2 ± 5.7	77.5 ± 6.6	63.3 ± 5.5	42.7	×

TABLE V
YOLO11N FP32, INT8 PTQ, AND INT8 QAT OVER THREE SEEDS AT $\tau = 0.5$

				INT8 PTQ					INT8 QAT				
Sparsity (%)	Param. red. (%)	GFLOPs	FP32 R (%)	MB	P (%)	R (%)	Eff_{MB}	Safe	MB	P (%)	R (%)	Eff_{MB}	Safe
0	0.0	6.50	86.4 ± 1.3	3.13	85.5 ± 5.4	85.6 ± 1.3	27.3	✓	2.92	84.6 ± 5.5	86.0 ± 1.7	29.5	✓
10	13.5	5.63	86.3 ± 3.7	2.81	89.0 ± 1.8	84.6 ± 4.6	30.1	✓	2.60	87.5 ± 3.6	86.3 ± 2.9	33.2	✓
20	24.9	4.92	88.0 ± 2.9	2.52	86.8 ± 1.5	87.6 ± 1.9	34.7	✓	2.31	83.8 ± 4.0	87.7 ± 2.2	37.9	✓
30	35.6	4.30	87.2 ± 2.1	2.25	87.6 ± 2.1	86.9 ± 2.1	38.6	✓	2.04	85.1 ± 3.5	86.8 ± 2.8	42.5	✓
40	44.8	3.74	84.4 ± 2.3	2.02	87.1 ± 1.5	84.9 ± 2.0	42.0	✓	1.81	87.1 ± 3.3	84.0 ± 4.9	46.3	✓
50	52.5	3.30	85.6 ± 2.3	1.82	85.8 ± 4.6	85.0 ± 2.8	46.7	✓	1.62	85.2 ± 2.3	87.6 ± 1.9	54.0	✓
60	59.8	2.84	88.1 ± 1.2	1.63	84.1 ± 1.1	87.7 ± 2.3	53.7	✓	1.44	84.1 ± 3.1	86.9 ± 0.7	60.5	✓
70	65.6	2.50	85.6 ± 1.4	1.48	84.2 ± 7.1	84.3 ± 1.2	56.8	✓	1.29	83.2 ± 5.8	84.8 ± 3.0	65.7	✓
80	70.4	2.24	79.8 ± 1.8	1.36	82.9 ± 2.2	78.9 ± 1.9	58.0	×	1.17	83.7 ± 1.7	80.2 ± 1.0	68.6	×
90	73.9	2.06	62.3 ± 1.7	1.27	80.2 ± 8.0	62.0 ± 2.2	48.9	×	1.08	78.2 ± 1.2	61.2 ± 0.5	56.7	×

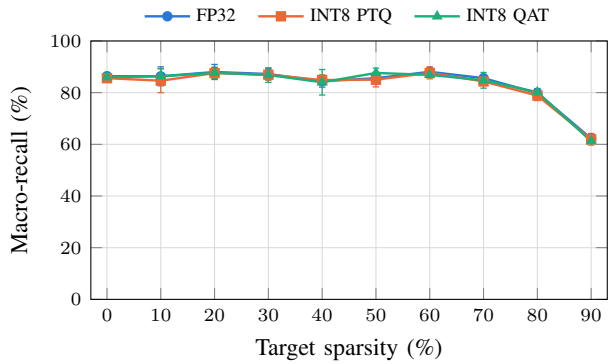


Fig. 3. YOLO11n macro-recall against target sparsity (mean ± SD, three seeds)

- *Quantization.* PTQ and QAT stay within 2.1 pp of FP32 at every sparsity up to 70%, with seed SD at most 4.9 pp. At 70% sparsity, PTQ gives $84.3 \pm 1.2\%$ in 1.48 MB and QAT gives $84.8 \pm 3.0\%$ in 1.29 MB. Both store the same 0.83 MB of INT8 weights. The PTQ file is larger because it quantizes every activation (795 quantize and dequantize nodes against 273), while QAT quantizes only convolution inputs. Both INT8 formats keep FP32 precision. Both fail at 80%, as FP32 does.

INT8 recall was stable only after a change of calibration. With plain MinMax ranges, PTQ recall varied by up to 32 pp between seeds of one configuration, and one seed reached 27.2% at 80% sparsity. Ranking survived: mAP@0.5 stayed at 0.87, and the confidences of the true class kept a 0.97 correlation with FP32 but halved in value. One rare outlier activation had set a range 12 times wider than typical, so the fixed threshold rejected correct detections. A moving average of per-frame extremes restored that seed to 77.5%. Class-balanced calibration frames did not help (30.4%). QAT also freezes its ranges after a short calibration, but it then trains the weights under those ranges, so the network adapts to any clipping. The earlier submission also quantized the class-logit outputs of the head, whose 40 to 150 unit range gives a step of 0.15 to 0.59 logit near the threshold. We keep those layers in floating point.

B. Phase II: Input-Side Optimization

Table VI and Fig. 4 report Phase II. Four findings stand out.

- *Resolution headroom.* Resizing the full frame holds recall down to 320 pixels (88.3% at 1.61 GFLOPs). At 160 pixels it fails the floor (80.7%) because yawning recall falls to 56.6%.

TABLE VI
FP32 YOLO11N BY INPUT METHOD AND RESOLUTION AT $\tau = 0.5$

Input (px)	GFLOPs	P (%)	R (%)	$F1$ (%)	mAP@0.5	mAP@0.5:0.95	R strict[a] (%)	Safe
Resize								
640	6.50	84.9 ± 4.0	87.4 ± 2.6	84.6 ± 2.0	92.5	55.6	87.4	✓
320	1.61	87.3 ± 1.4	88.3 ± 1.2	87.4 ± 0.9	93.3	58.8	88.3	✓
160	0.40	88.3 ± 1.5	80.7 ± 1.6	83.1 ± 0.8	90.8	54.3	80.7	×
80	0.14	87.3 ± 0.6	72.6 ± 1.7	77.1 ± 1.3	84.3	47.3	72.6	×
Crop								
640	6.50	87.1 ± 1.7	81.5 ± 2.3	83.2 ± 1.0	87.8	58.8	72.6	✓
320	1.61	87.5 ± 2.5	84.7 ± 0.8	85.4 ± 1.4	89.0	61.4	75.8	✓
160	0.40	87.8 ± 1.0	82.3 ± 3.1	84.2 ± 1.7	86.6	58.2	73.5	✓
80	0.14	86.3 ± 4.6	75.7 ± 2.6	79.0 ± 2.1	84.5	53.2	67.0	×
Collage								
640	6.50	86.9 ± 2.4	90.5 ± 2.2	87.9 ± 1.2	92.3	66.8	89.0	✓
320	1.61	86.6 ± 0.8	93.2 ± 0.7	89.2 ± 0.7	92.6	67.9	91.1	✓
160	0.40	82.5 ± 1.7	88.7 ± 1.8	84.8 ± 1.5	87.1	59.8	87.1	✓
80	0.14	82.9 ± 2.8	75.3 ± 3.0	76.7 ± 1.4	80.0	49.9	73.5	×

[a]A box counts only if at least 50% of it stays visible.

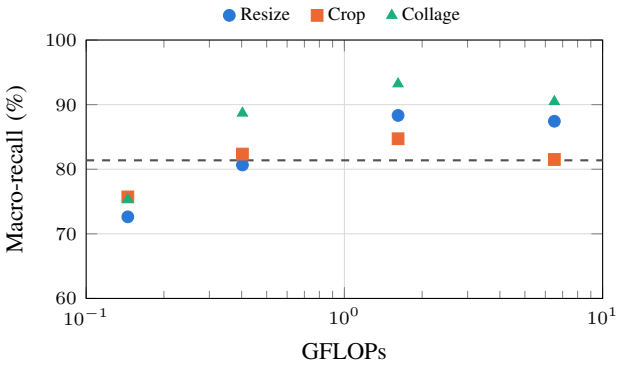


Fig. 4. FP32 macro-recall against compute for each input method. The dashed line is the floor

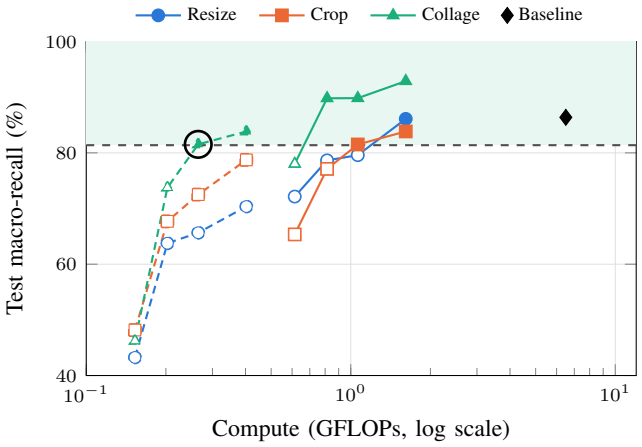


Fig. 5. Test macro-recall of every INT8 candidate against compute, with solid lines at 320 pixels and dashed lines at 160 pixels. The shaded band clears the floor, hollow markers fail it, and the ringed point is the recommended configuration

- *Crop.* Magnifying the face lifts yawning recall at 160 pixels from 56.6% to 87.9%. The window cuts peripheral objects, though: phone-use recall stays near 70% at every resolution. When a box must stay at least half visible to count, the crop falls below the floor at every resolution.
- *Collage.* The collage gives the highest recall at every resolution from 640 to 160 pixels: 90.5%, 93.2%, and 88.7%, with F1 of 0.892 at 320 pixels. It holds 87.1% to 91.1% under the stricter visibility rule. The overlapping tiles show the face and hand zones twice, which helps most for yawning (96.0% at 320 pixels against 87.9% for resize). The first submission scored each tile fragment as a separate object, which hid this result.
- *Lower limit.* No method holds the floor at 80 pixels.

C. Phase III: System Co-Design

Table VII, Table VIII, and Fig. 5 report Phase III. Four findings stand out.

- *Pruning at reduced input.* Pruning hurts more when the input is small. At 320 pixels the collage holds the floor up to 50% sparsity (89.8%), the crop up to 30% (81.5%), and resize only unpruned (79.6% at 30%). At 70% sparsity all three methods fail at 320 pixels (65.4% to 78.0%), although the same sparsity holds the floor at 640 pixels. At 160 pixels resize and crop fail even unpruned (70.4% and 78.7%). On the 160-pixel collage, 30% sparsity still holds (81.5%), 50% does not (73.8%), and 70% collapses (46.2%).
- *Validation does not transfer.* The rule in (1) selects the 160-pixel collage with 50% sparsity, at 0.20 GFLOPs, which is 1.3 pp above the baseline on validation. On the test drivers it falls 12.6 pp below the baseline because yawning recall drops to 31.3% (95% interval 21% to 47%). Three validation subjects with about 30 frames per rare class cannot certify a configuration that changes both input scale and network width.
- *Recommended configuration.* Table VII ranks every candidate by compute, then memory, and Fig. 5 shows the same ranking against the floor. Every candidate cheaper than 0.27 GFLOPs fails. The first that holds the floor is the 160-pixel collage with 30% sparsity and INT8

TABLE VII
TEST MACRO-RECALL (%) OF INT8 PTQ MODELS ACROSS INPUT, RESOLUTION, AND SPARSITY

	Sparsity			
	0%	30%	50%	70%
320 px: GFLOPs / MB	1.61 / 3.01	1.06 / 2.13	0.81 / 1.70	0.61 / 1.36
Resize	86.1 ± 1.0 ✓	79.6 ± 6.7 ×	78.7 ± 4.1 ×	72.1 ± 3.7 ×
Crop	83.8 ± 1.7 ✓	81.5 ± 3.6 ✓	77.1 ± 7.1 ×	65.4 ± 5.3 ×
Collage	92.9 ± 0.2 ✓	89.8 ± 1.5 ✓	89.8 ± 0.5 ✓	78.0 ± 0.9 ×
160 px: GFLOPs / MB	0.40 / 2.98	0.27 / 2.10	0.20 / 1.67	0.15 / 1.33
Resize	70.4 ± 1.8 ×	65.7 ± 3.5 ×	63.8 ± 0.5 ×	43.3 ± 4.3 ×
Crop	78.7 ± 3.3 ×	72.5 ± 0.7 ×	67.7 ± 8.4 ×	48.2 ± 6.7 ×
Collage	83.8 ± 4.8 ✓	**81.5 ± 4.3** ✓	73.8 ± 4.8 ×^{S}	46.2 ± 2.5 ×
Baseline, 640 px, FP32	86.4 ± 1.3 at 6.50 GFLOPs / 10.12 MB			

Mean ± SD over three seeds. ✓: holds the 81.4% floor. Shaded: the cheapest configuration that holds it. S: selected on validation by (1).

TABLE VIII
TEST RECALL (%) WITH CLIP-BOOTSTRAP 95% INTERVALS

Class	**Baseline**	**Selected (S)**	**Recommended (R)**
Yawning	80.8 (64–94)	31.3 (21–47)	49.5 (37–65)
Hand-over-mouth	92.9 (67–100)	90.5 (71–98)	94.0 (70–100)
Drinking	89.3 (79–100)	89.9 (85–95)	97.6 (97–98)
Phone use	82.6 (50–99)	83.4 (50–100)	85.0 (54–100)
Macro	86.4 (73–95)	73.8 (63–81)	81.5 (69–88)

PTQ: $81.5 \pm 4.3\%$ macro-recall at 0.27 GFLOPs and 2.10 MB, which is $24.5\times$ less compute and $4.8\times$ less storage than the FP32 ONNX baseline. It is the least costly configuration this study supports, not a comfortable one: it clears the floor by 0.1 pp, its 95% interval spans 69.5% to 88.5%, and yawning recall falls to 49.5%.

- *More recall for more compute.* The unpruned 160-pixel collage with INT8 holds 83.8% at 0.40 GFLOPs and 2.98 MB. Pruning the 320-pixel collage to 50% keeps $89.8 \pm 0.5\%$ at 0.81 GFLOPs and 1.70 MB ($8.0\times$ less compute), and stays above the floor under the stricter visibility rule (86.8%) and the tuned threshold (86.7%). The 320-pixel collage with INT8 gives the highest recall, $92.9 \pm 0.2\%$ at 1.61 GFLOPs and 3.01 MB ($4.0\times$ less compute), 6.5 pp above the baseline (95% interval 0.9 to 13.6 pp), with every class at 82.6% or more. It is the choice when the margin matters more than the last factor of compute.

Two further checks qualify these results. The recommended configuration falls just below the floor under the stricter visibility rule (79.3%) and under a validation-tuned threshold (80.0%), so its pass at $\tau = 0.5$ is narrow. The 320-pixel collage keeps 91.1% and 89.4% under the same checks, and the selected 50% configuration fails under both. Because the recommendation uses test results, we report it as a recommendation and not as a validated selection. The two INT8 sizes in the first submission, 1.31 and 1.19 MB for one network, differ only by the anchor tables that a fixed-shape graph stores: 0.128 MB at 640 pixels and 0.032 MB at 320 pixels.

D. Limitations and Future Work

The test set holds three drivers and each rare class comes from five clips, so our intervals are wide, and the failed selection in Phase III shows that three validation drivers cannot certify a pruned low-resolution model. We did not measure latency, power, or memory on automotive hardware, where INT8 support changes the real gains, and an integer-only accelerator would also have to run the floating-point head outputs. The crop and collage also assume a fixed camera mount. Future work should add drivers and cabins, select configurations with cross-validation over subjects, and measure the recommended pipeline on target hardware.

V. CONCLUSION

This paper asked whether model compression and input preprocessing can be co-designed for edge driver monitoring without violating a safety recall floor. We evaluated structured ℓ_1-norm pruning with INT8 PTQ and QAT on YOLO11n and YOLO26n, together with resize, crop, and collage inputs from 640 to 80 pixels, under one protocol on the same subject-disjoint frames and boxes. Three conclusions follow. First, the model has real headroom at full resolution, because YOLO11n holds the floor up to 70% sparsity and INT8 keeps that recall at 1.3 to 1.5 MB once ranges are calibrated with a moving average. Second, the input matters as much as the model, because the overlapping collage gives the highest recall at every resolution down to 160 pixels, while a static crop loses peripheral cues such as phone use. Third, the two levers combine well only in moderation. The cheapest configuration that holds the floor, a 160-pixel collage with 30% sparsity and INT8, needs $24.5\times$ less compute than the baseline but passes only narrowly, while the 320-pixel collage with INT8 keeps 92.9% recall at $4.0\times$ less compute. Heavier pruning at reduced input did not transfer from validation to test drivers, so input reshaping and quantization are the safer levers, and on-device tests with more drivers are the next step.

REFERENCES

[1] J. Guo, W. Ouyang, and D. Xu, "Multi-dimensional pruning: A unified framework for model compression," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 1508–1517.

[2] A. Vina, "Pruning and quantization in computer vision: A quick guide," Ultralytics Blog, Jul. 11, 2025. Accessed: Sep. 29, 2026. [Online]. Available: https://www.ultralytics.com/blog/pruning-and-quantization-in-computer-vision-a-quick-guide

[3] Y. He, P. Liu, L. Zhu, and Y. Yang, "Filter pruning by switching to neighboring CNNs with good attributes," *IEEE Trans. Neural Netw. Learn. Syst.*, pp. 1–13, 2022, doi: 10.1109/TNNLS.2022.3149332.

[4] Z. Wang, C. Li, and X. Wang, "Convolutional neural network pruning with structural redundancy reduction," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 14913–14922.

[5] Z. Li, B. Ni, T. Li, and W. Gao, "Residual quantization for low bit-width neural networks," *IEEE Trans. Multimedia*, vol. 25, pp. 214–227, 2023, doi: 10.1109/TMM.2021.3124095.

[6] S. Li *et al.*, "Distilling a powerful student model via online knowledge distillation," *IEEE Trans. Neural Netw. Learn. Syst.*, pp. 1–10, 2022, doi: 10.1109/TNNLS.2022.3152732.

[7] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016.

[8] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, "Pruning filters for efficient ConvNets," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017.

[9] Y. He, X. Zhang, and J. Sun, "Channel pruning for accelerating very deep neural networks," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 1389–1397.

[10] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang, "Learning efficient convolutional networks through network slimming," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 2736–2744.

[11] G. Fang, X. Ma, M. Song, M. B. Mi, and X. Wang, "DepGraph: Towards any structural pruning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 16091–16101.

[12] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 2704–2713.

[13] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," arXiv:1806.08342, 2018.

[14] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 7464–7475.

[15] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "YOLOv10: Real-time end-to-end object detection," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024.

[16] R. Khanam and M. Hussain, "YOLOv11: An overview of the key architectural enhancements," arXiv:2410.17725, 2024.

[17] R. Sapkota, R. H. Cheppally, A. Sharda, and M. Karkee, "YOLO26: Key architectural enhancements and performance benchmarking for real-time object detection," arXiv:2509.25164, 2025.

[18] F. O. Unel, B. O. Ozkalayci, and C. Cigla, "The power of tiling for small object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, 2019, pp. 582–591.

[19] M. Najibi, B. Singh, and L. S. Davis, "AutoFocus: Efficient multi-scale inference," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 9745–9755.

[20] J. D. Ortega, N. Kose, P. Cañas, M.-A. Chao, A. Unnervik, M. Nieto, O. Otaegui, and L. Salgado, "DMD: A large-scale multi-modal driver monitoring dataset for attention and alertness analysis," in *Proc. Eur. Conf. Comput. Vis. Workshops (ECCVW)*, 2020, pp. 387–405.

[21] Y. Abouelnaga, H. M. Eraqi, and M. N. Moustafa, "Real-time distracted driver posture classification," arXiv:1706.09498, 2017.

[22] A. Kashevnik, R. Shchedrin, C. Kaiser, and A. Stocker, "Driver distraction detection methods: A literature review and framework," *IEEE Access*, vol. 9, pp. 60063–60076, 2021.

[23] Euro NCAP, "Safe Driving: Driver Engagement Protocol," Version 1.2, Euro NCAP, Leuven, Belgium, Jul. 2026. [Online]. Available: https://cdn.euroncap.com/cars/assets/Euro_NCAP_Protocol_Safe_Driving_Driver_Engagement_v1_2_ebce03a443.pdf

[24] *Road Vehicles: Driver Readiness and Intervention Management, Part 1: Partial Automation (Level 2)*, ISO/TS 5283-1:2025, International Organization for Standardization, Geneva, Switzerland, Oct. 2025. [Online]. Available: https://www.iso.org/standard/85035.html